

Automatic Fraud Detection

Plan du pipeline de détection de fraude en temps réel — ETL with Airflow

Emeline ROBLOT · Jedha · septembre 2026 · github.com/emelineroblot/automatic-fraud-detection

1 tx / min

collectée depuis l'API et scorée
par Airflow

< 30 s

entre la réception d'une
transaction et l'alerte Discord

0,805

PR-AUC du RandomForest retenu
— split temporel, 0,39 % de
fraudes

19

ressources AWS déclarées en
Terraform (EC2, S3, IAM, réseau,
secrets)

Deux besoins métier, deux pipelines Airflow

Besoin 1 — être notifié dès qu'une fraude est détectée

*DAG fraud_realtime · schedule * * * * * · run ≈ 18 s*



extract : GET de l'API (JSON double-encodé), 2 retries

transform : normalisation, current_time (ms) → trans_time UTC

predict : modèle chargé depuis MLflow
(models:/fraud_detection@production)

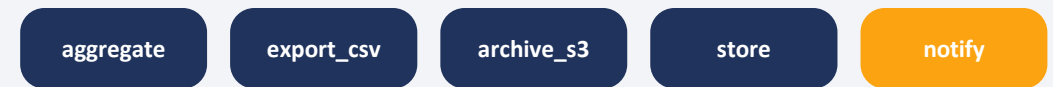
load : INSERT ... ON CONFLICT (trans_num) DO NOTHING RETURNING —
dédoublonnage

check_fraud : branche → alert (embed Discord + trace fraud_alerts) ou
no_fraud

Garde-fous : max_active_runs = 1, catchup = False, dagrun_timeout 5
min

Besoin 2 — chaque matin, les paiements et fraudes de la veille

*DAG fraud_daily_report · schedule 0 6 * * * (Europe/Paris)*



aggregate : volumes, montants, top catégories / états, répartition
horaire

précision et rappel du jour vs label fourni par l'API (vérité terrain)

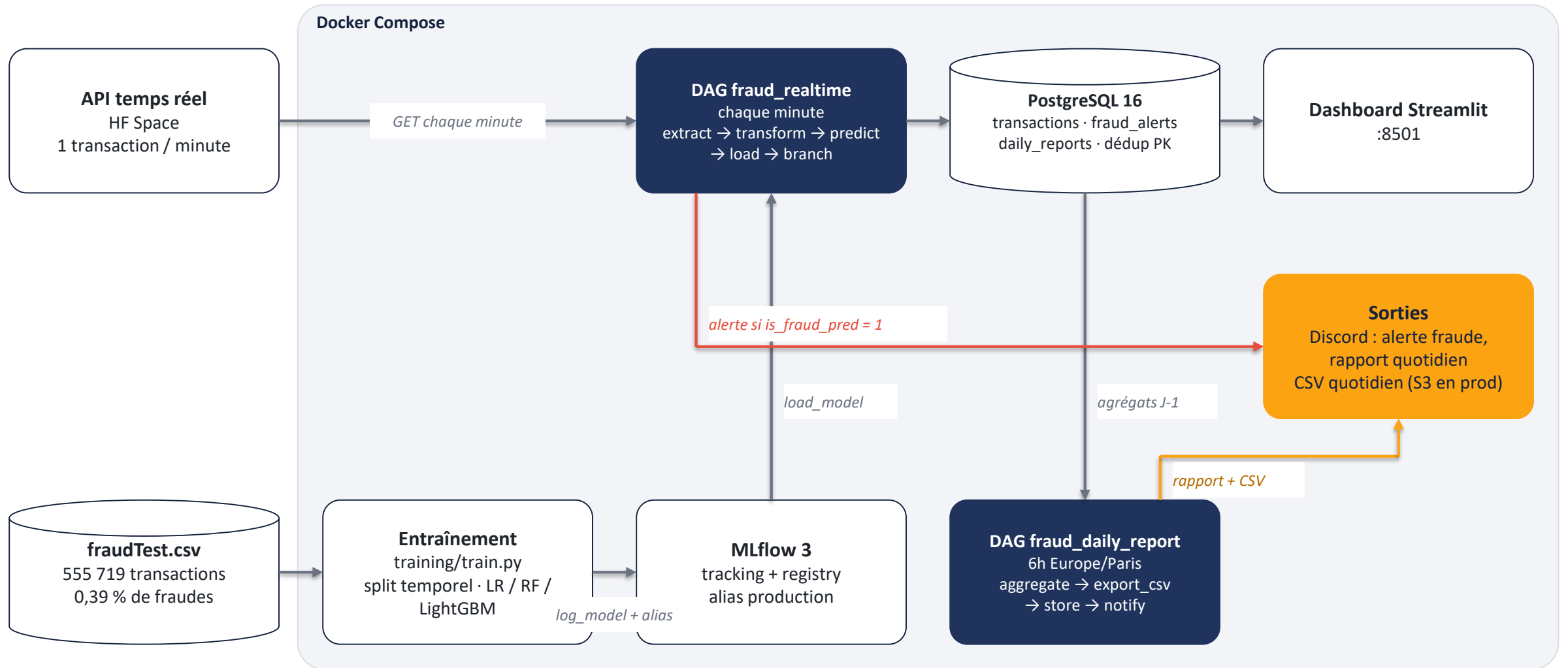
export_csv : reports/transactions_<date>.csv → archive_s3 :
s3://.../reports/

store : table daily_reports (agrégats + payload JSONB)

notify : embed Discord récapitulatif

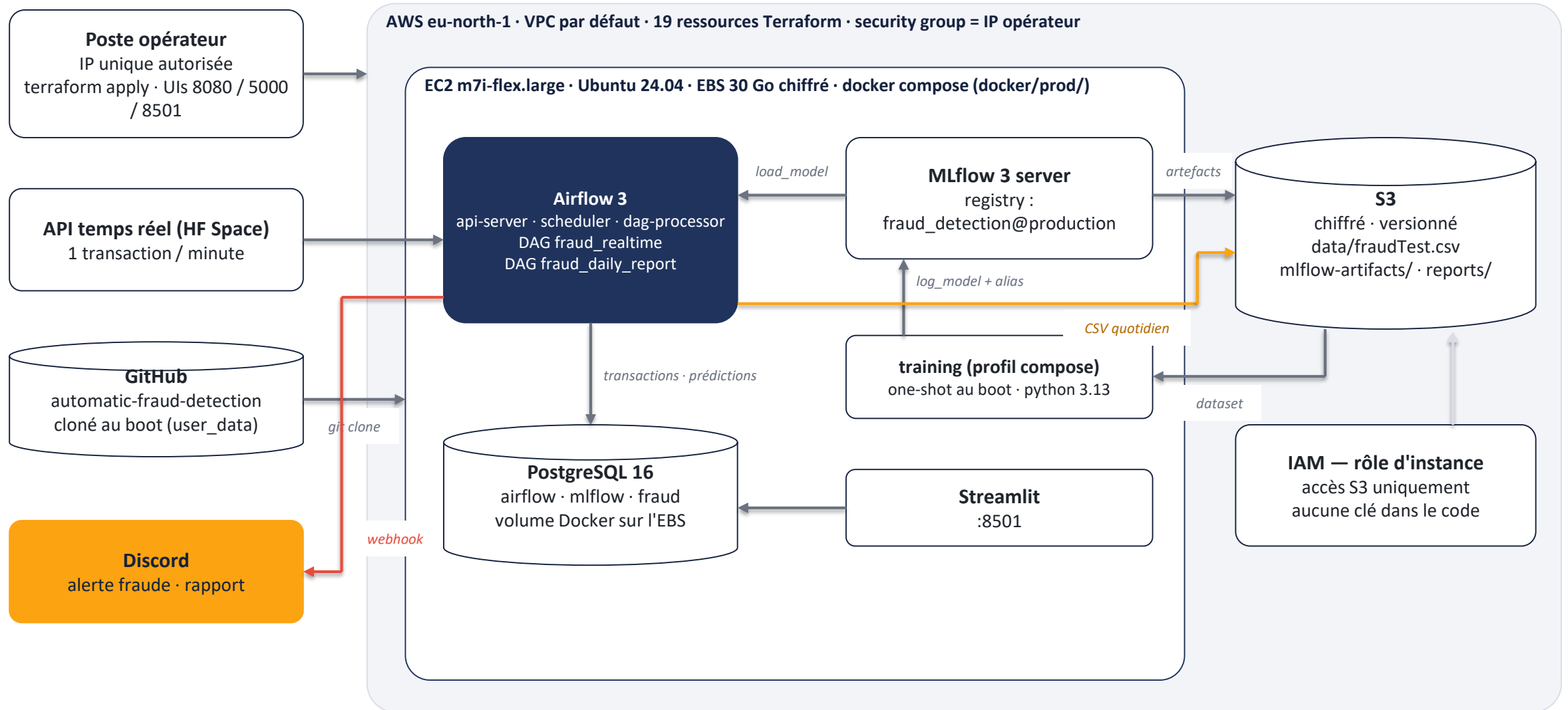
Paramètre report_date pour rejouer une journée (démonstration)

Architecture logique



Un orchestrateur (Airflow), un entrepôt (PostgreSQL), un registre (MLflow) : le modèle est un artefact versionné ; le preprocessing (FeatureBuilder) est dans le pipeline sklearn loggé — une ligne brute de l'API traverse exactement les mêmes transformations qu'une ligne du CSV.

Déploiement en production — AWS eu-north-1 (Terraform)



RDS PostgreSQL était la cible : bloqué par le quota du plan gratuit du compte (1 instance, déjà utilisée). Coût ≈ 2,5 \$ / jour · terraform destroy après la soutenance.

Choix d'architecture — et ce qui a été écarté

Choix	Alternative écartée	Pourquoi
Airflow seul, DAG à la minute	Kafka + producer / consumer	l'API produit 1 transaction par minute : Kafka ajouterait 2 services pour une latence identique. Pertinent si le débit monte ou si plusieurs consommateurs lisent le flux.
Modèle chargé depuis le registry MLflow	API FastAPI de scoring	un seul client (le DAG) ; l'alias production découple déjà entraînement et exploitation. Une API se justifie avec un second consommateur.
Preprocessing dans le pipeline sklearn	code de features dupliqué dans le DAG	un seul artefact, predict_proba sur du brut : aucune divergence training / inférence possible (exigence « réutilisable »).
PostgreSQL	warehouse analytique (BigQuery, Snowflake)	1 440 lignes / jour : clé primaire pour le dédoublonnage, JSONB, agrégats SQL. Réplication vers un warehouse au-delà.
Webhook Discord	e-mail SMTP	5 lignes de code, message riche, visible en démo ; le métier « a juste besoin d'une notification ».
Terraform + docker compose sur EC2	MWAA, ECS, RDS	reproductible en une commande, ≈ 2,5 \$ / jour ; RDS bloqué par le quota du compte, décrit comme cible.